

ATACAR

SESIÓN 09 · UNIDAD II · CIERRE

EL CÓDIGO ES PÚBLICO, CUSTODIA VALOR Y NO SE PARCHA

Hoy cambiamos de lado: para defender un contrato hay que saber atacarlo. Y al final, un parcial: encontrar, explotar y corregir.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

LO DE HOY

CÓMO VAMOS A USAR LAS TRES HORAS

Cierra la Unidad II. Primero la taxonomía completa y un robo en vivo; después el CTF Ethernaut y la auditoría cruzada; y de últimas, 90 minutos de parcial individual.

A	**TAXONOMÍA Y DEFENSAS** Las vulnerabilidades más caras de la historia, una explotada en vivo, y el ciclo de vida seguro.	~55 min
B	**ETHERNAUT + AUDITORÍA** Cinco niveles del CTF y auditoría cruzada #1 entre equipos.	~35 min
C	**PARCIAL PRÁCTICO** Encontrar, explotar y corregir. Individual, libro abierto.	90 min

AL SALIR DE AQUÍ

Identificar, explotar y remediar las vulnerabilidades más frecuentes de los contratos inteligentes.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

01 ¿Por qué la seguridad aquí es distinta de la de una aplicación web?

02 ¿Cómo se roba un contrato con reentrancy, línea por línea?

03 ¿Qué defensas existen, y por qué ninguna sola basta?

04 ¿Cómo se lee un contrato ajeno buscando el fallo?

RA5 · Identificar y remediar vulnerabilidades típicas mediante análisis manual y herramientas automáticas.

SIGLAS Y TÉRMINOS DE HOY

REENTRANCY *reentrada*

Un contrato externo vuelve a llamar antes de que el primero termine de actualizar su estado.

CEI *Checks-Effects-Interactions*

Validar, actualizar el estado propio y solo al final llamar hacia afuera.

TX.ORIGIN *origen de la transacción*

La cuenta que firmó. NUNCA para permisos: quien llama directo es msg.sender.

DELEGATECALL *llamada delegada*

Ejecuta el código de otro contrato con el almacenamiento del propio. Poderosa y peligrosa.

MEV *Maximal Extractable Value*

Ganancia que un productor de bloques extrae reordenando, insertando o censurando transacciones.

ORÁCULO *oracle*

Servicio que trae datos externos a la cadena. Su manipulación es un vector de ataque frecuente.

SLITHER *analizador estático*

Herramienta que revisa el código sin ejecutarlo y marca patrones peligrosos.

PROXY *contrato actualizable*

Separa el almacenamiento (proxy) de la lógica (implementación) vía delegatecall, para poder cambiar el código.

A

APROXIMADAMENTE 55 MINUTOS

TAXONOMÍA Y DEFENSAS

Cada vulnerabilidad de esta lista costó dinero real. Varias, cientos de millones. Ninguna era código roto: era código que hacía exactamente lo que decía.

TRES COSAS QUE NO PASAN EN UNA APP NORMAL

UNA APLICACIÓN WEB

- Si hay un error, se publica un parche.
- El código corre en un servidor privado.
- El dinero está en un banco con reversos y soporte.

UN CONTRATO

- El código desplegado es inmutable: no hay parche.
- Es público: cualquiera lo lee y lo estudia.
- Custodia valor directamente, y las transferencias no se revierten.

En la web, un atacante busca una puerta mal cerrada. Aquí tiene los planos del edificio, tiempo ilimitado, y si entra se lleva la caja fuerte sin que nadie pueda perseguirlo.

Por eso el orden es: escribir menos, reutilizar código auditado, probar los bordes (Sesión 8) y auditar antes de tocar dinero.

LA TAXONOMÍA DE HOY: OCHO VECTORES

VULNERABILIDAD	EN UNA FRASE	CASO CÉLEBRE (VERIFICADO)
Reentrancy	Vuelven a entrar antes de que actualices el estado.	The DAO, jun 2016 · ~3,6 M ETH
Control de acceso	Falta un permiso, o se comprueba mal (tx.origin).	Parity multisig, jul 2017
delegatecall no protegido	Código ajeno escribe en tu almacenamiento.	Parity, nov 2017 · 513.774 ETH congelados
Desbordamiento aritmético	Un número da la vuelta. Resuelto en 0.8.	BeautyChain (BEC), abr 2018
Aleatoriedad on-chain	El azar es predecible o influenciable.	SmartBillions, oct 2017 · ~400 ETH
Front-running / MEV	Ven tu transacción y se adelantan.	DEX, a diario · «Flash Boys 2.0»
Manipulación de oráculos	Falsean el precio del que dependes.	Mango Markets, oct 2022 · >110 M USD
DoS por bucle o receptor	Un dato o una cuenta bloquean a todos.	King of the Ether, feb 2016

Reentrancy la explotamos en vivo. Las demás, con su caso real. Todas pueden salir en el parcial.

UN BANCO QUE PARECE CORRECTO

```
contract BancoVulnerable {
    mapping(address => uint256) public saldos;

    function depositar() external payable {
        saldos[msg.sender] += msg.value;
    }

    function retirar() external {
        uint256 monto = saldos[msg.sender];
        require(monto > 0, "sin saldo");

        (bool ok, ) = payable(msg.sender).call{value: monto}("");
        require(ok, "envio fallido");

        saldos[msg.sender] = 0;     // <-- el saldo se limpia AL FINAL
    }
}
```

- Lleva la cuenta de cada depósito.
- Solo deja retirar lo propio.
- Comprueba que el envío no falle.
- Pone el saldo en cero.

LA PREGUNTA ANTES DE SEGUIR

Todo parece bien. ¿En qué línea está el fallo? Pista: piensen qué pasa DURANTE el call, antes de la última línea.

EL ENVÍO CEDE EL CONTROL AL ATACANTE

call{value: monto} no es solo mover ETH. Si el receptor es un contrato, EJECUTA su función receive(). Y ahí, el atacante toma el control con el saldo todavía sin limpiar.

01	LÍNEA 1 DEL ATAQUE	El banco ejecuta call y le envía el ETH al contrato atacante.
02	EL CONTROL SALTA	Se ejecuta receive() del atacante — antes de que el banco llegue a saldos = 0.
03	EL ATACANTE REENTRA	Desde receive() vuelve a llamar retirar(). El banco aún cree que tiene saldo.

LA REGLA QUE SE VIOLÓ

Interacción (enviar) antes que efecto (limpiar el saldo). Es exactamente al revés de Checks-Effects-Interactions.

TRECE LÍNEAS QUE VACÍAN EL BANCO

```
contract Atacante {
    IBanco banco;  uint256 cebo;

    function atacar() external payable {
        cebo = msg.value;
        banco.depositar{value: msg.value}();   // deposita 1
        banco.retirar();                        // pide su retiro
    }

    receive() external payable {                // el banco le paga...
        if (address(banco).balance >= cebo) {
            banco.retirar();                    // ...y vuelve a entrar
        }
    }
}
```

1 → 11

deposita 1 ETH, se lleva los 11 del banco

Verificado en test/s09: tres clientes honestos tenían 10 ETH; el atacante depositó 1 y salió con 11. El banco quedó en cero, y los saldos honestos quedaron impagables.

THE DAO, JUNIO DE 2016

The DAO era un fondo de inversión colectivo gobernado por contrato. Recaudó una de las mayores sumas de su tiempo (del orden de 150 millones de dólares). Tenía exactamente este error.

~3,6 M ETH

drenados por reentrancy en unas horas

≈ 5 %

de todo el ETH que existía entonces

1 línea

el orden invertido de dos instrucciones

La respuesta partió a Ethereum en dos: la mayoría hizo una bifurcación dura para revertir el robo (la cadena que hoy llamamos Ethereum); una minoría se negó por principio y siguió en la cadena original (Ethereum Classic). Es el ejemplo de la Sesión 4 sobre que la inmutabilidad también es una convención social.

DOS CINTURONES PARA EL MISMO RIESGO

1 · CHECKS-EFFECTS-INTERACTIONS

```
saldos[msg.sender] = 0;   // primero
(bool ok, ) = ...call{value: monto}("");
```

Cuando el atacante recibe el control, su saldo ya es cero. Reentrar no sirve: no hay nada que cobrar.

2 · REENTRANCYGUARD

```
function retirar() external nonReentrant { ... }
```

Un cerrojo que se cierra al entrar y se abre al salir. La segunda entrada revierte, aunque alguien olvide el orden.

[-] POR QUÉ AMBAS, SI UNA BASTA

CEI es la corrección de fondo y no cuesta gas extra. El guard es defensa en profundidad: protege del error humano futuro, cuando alguien modifique la función dentro de seis meses y rompa el orden sin darse cuenta. En seguridad no se apuesta todo a una sola barrera.

¿ENTENDIERON EL PATRÓN?

Un compañero dice: «lo arreglo poniendo require(msg.sender == tx.origin) para que solo entren personas, no contratos». ¿Resuelve la reentrancy?

LA RESPUESTA CORTA

No. Y además introduce OTRO problema: rompe la compatibilidad con billeteras-contrato (multisig, smart accounts) y usa tx.origin, que es justo lo que veremos que nunca se debe usar para lógica.

LO QUE SÍ RESUELVE

Poner el efecto antes de la interacción (CEI). El guard es el refuerzo. La reentrancy es un problema de ORDEN, no de quién llama.

EL PERMISO QUE MIRA A LA CUENTA EQUIVOCADA

```
require(tx.origin == admin, "solo admin");    // MAL
require(msg.sender == admin, "solo admin");   // BIEN
```

TX.ORIGIN

La cuenta que FIRMÓ la transacción, al principio de toda la cadena de llamadas. Si el admin firma una transacción a un contrato malicioso, ese contrato llama al vault y tx.origin sigue siendo el admin: la comprobación pasa.

MSG.SENDER

Quien llama DIRECTAMENTE a esta función. Si el contrato malicioso llama al vault, msg.sender es el contrato malicioso, no el admin: la comprobación falla, como debe ser.

REGLA SIN EXCEPCIONES

tx.origin nunca se usa para autorizar. Ante la duda, msg.sender — o mejor, Ownable / AccessControl de OpenZeppelin.

EL CÓDIGO AJENO QUE ESCRIBE EN TU CASA

delegatecall ejecuta el código de OTRO contrato pero con TU almacenamiento y TU saldo. Es lo que hace posibles los proxies (A.20). Sin protección, es una de las llaves maestras más peligrosas de la EVM.

01	LA BIBLIOTECA COMPARTIDA	Cientos de multisigs delegaban su lógica en una única biblioteca de código. Esa biblioteca quedó sin inicializar.
02	CUALQUIERA LA INICIALIZA	Una función de inicialización sin control de acceso permitió a un desconocido declararse dueño de la biblioteca.
03	Y LUEGO LA MATA	Siendo dueño, invocó selfdestruct sobre la biblioteca. Al destruirse el código del que TODOS dependían por delegatecall...
04	513.774 ETH CONGELADOS	...los 587 monederos que la usaban quedaron sin lógica: nadie pudo volver a mover sus fondos. No fue robo: fue un ladrillo.

LA DEFENSA

No usar delegatecall salvo para proxies bien entendidos. Nunca hacia una dirección que un usuario elija. Inicializar con un initializer protegido.

UN NÚMERO QUE DA LA VUELTA

```
// Solidity < 0.8, sin SafeMath:
uint8 x = 255;
x = x + 1;       // -> 0    (envolvente, en silencio)

uint256 saldo = 20;
saldo = saldo - 21;    // -> 2^256 - 1   (underflow gigante)
```

QUÉ PASABA ANTES DE 0.8

La aritmética envolvía sin avisar. Un require(saldo - monto >= 0) con uint es SIEMPRE cierto: una resta de uint nunca es negativa. Así se saltaban chequeos de saldo.

QUÉ CAMBIÓ EN SOLIDITY 0.8

Desde 0.8.0 (dic 2020) se verifica por defecto: over/underflow REVIERTEN. El riesgo hoy vive en los bloques unchecked { ... }, para ahorrar gas.

⚠ **EL CASO · BEAUTYCHAIN (BEC)**

En abril de 2018, el «batchOverflow» (CVE-2018-10299) acuñó tokens astronómicos de la nada por un overflow en batchTransfer. Es una de las razones por las que 0.8 revierte solo.

NO HAY AZAR DENTRO DE LA CADENA

Todo lo que un contrato puede leer —block.timestamp, block.prevrandao, blockhash— es público y, en parte, influenciable por el proponente del bloque. Un contrato atacante calcula el mismo «azar» en la misma transacción antes de apostar.

```
// Loteria ingenua: el "azar" es una cuenta pública
uint256 premio = uint256(keccak256(
    abi.encodePacked(block.timestamp, block.prevrandao))) % 10;
// El atacante ejecuta el MISMO cálculo en su contrato y
// solo juega cuando premio le favorece.
```

EL CASO · SMARTBILLIONS, Y LA DEFENSA

Una lotería on-chain perdió ~400 ETH (oct 2017) porque su «azar» se leía del estado y se podía predecir. Solución: un oráculo de aleatoriedad verificable (Chainlink VRF) o un esquema commit-reveal (comprometer un valor cifrado y revelarlo después). Nunca variables de bloque. «Poner más variables de bloque» NO arregla nada.

TODAS TUS INTENCIONES SON PÚBLICAS

Antes de confirmarse, tu transacción vive en el mempool a la vista de todos. Quien arma el bloque puede colar la suya antes, después, o reordenar, para ganar a tu costa. Es el «valor máximo extraíble» (MEV).

CÓMO TE AFECTA

- Un bot ve tu compra grande en un DEX y compra antes para venderte más caro (sandwich).
- Ve tu solución a un puzzle premiado y la copia con más gas.
- Reordena liquidaciones para quedarse el bono.

LAS DEFENSAS

- Límites de deslizamiento (slippage) en cada swap.
- Revelar en dos pasos (commit-reveal) lo que no debe verse antes.
- Subastas por lotes; mempools privados.
- Nunca suponer el orden de llegada.

No es un bug de tu contrato: es una propiedad del sistema. Se diseña PARA él, no contra él.

SI DEPENDES DE UN PRECIO, PUEDEN MOVERLO

Un contrato que lee el precio «spot» de un solo mercado (un DEX) se puede engañar moviendo ese precio, aunque sea por un instante, con un préstamo relámpago dentro de la misma transacción.

01	PIDE PRESTADO ENORME	Un flash loan le da millones sin garantía, a devolver en la misma transacción.
02	MUEVE EL PRECIO SPOT	Compra masivamente el activo en el mercado que el oráculo lee: el precio se dispara.
03	EXPLOTA AL QUE LO LEE	El contrato-víctima ve ese precio inflado y le presta o le paga de más contra ese valor.
04	DEVUELVE Y SE VA	Devuelve el préstamo, deshace la posición y se queda la diferencia. Todo en un bloque.

EL CASO · MANGO MARKETS, Y LA DEFENSA

Mango Markets perdió más de 110 M USD (oct 2022) por manipulación de su oráculo de precio. Defensa: precios promediados en el tiempo (TWAP), varios oráculos independientes, servicios como Chainlink. Nunca un solo precio spot.

UN SOLO DATO PUEDE BLOQUEAR A TODOS

DOS POR RECEPTOR QUE REVIERTE

Si el contrato PAGA en un bucle y una de las cuentas es un contrato que revierte al recibir (o consume todo el gas), el pago a TODOS los demás falla. King of the Ether (feb 2016) se atascó así: send() a una wallet-contrato falló y el retorno no se comprobó.

DOS POR BUCLE NO ACOTADO

Recorrer un arreglo que crece sin límite (la lista de participantes de la S7) puede superar el límite de gas de un bloque: la función se vuelve imposible de ejecutar. El contrato queda vivo pero inútil.

[-] LA DEFENSA: EL PATRÓN DE RETIRO (S7)

No EMPUJES los pagos (push): deja que cada quien RETIRE lo suyo (pull). Así un receptor que revierte solo se afecta a sí mismo. No recorras arreglos ilimitados; no dependas de que un envío externo funcione. Es el mismo patrón «pull over push» de la Sesión 7.

CLASIFIQUEN ESTOS TRES FALLOS

SÍNTOMA EN EL CÓDIGO	¿QUÉ VULNERABILIDAD ES?
`Una función de retiro que envía ETH y después descuenta el saldo.`	Reentrancy (orden CEI violado).
`require(tx.origin == owner) para autorizar una función crítica.`	Control de acceso mal implementado.
`Un ganador elegido con keccak256(block.timestamp) % n.`	Aleatoriedad on-chain manipulable.

CÓMO SE RESPONDE EN EL PARCIAL

Tapen la segunda columna y pídanle a la clase nombrarlas en voz alta, con la LÍNEA exacta. En el parcial se pide justo esto: nombre + severidad + ubicación. Nombrar sin ubicar vale la mitad. Estos tres son ejemplos de método; el contrato del parcial es otro.

LA SEGURIDAD NO ES UN PASO: ES UNA CADENA

01	ESCRIBIR MENOS	Cada línea propia es superficie de ataque. Reutilizar OpenZeppelin auditado en lugar de reescribir.
02	PROBAR LOS BORDES	La suite de la Sesión 8, con pruebas que atrapan errores, no solo que dan cobertura.
03	ANÁLISIS ESTÁTICO	Slither revisa el código sin ejecutarlo y marca patrones peligrosos en segundos.
04	FUZZING E INVARIANTES	Miles de entradas al azar buscando romper una invariante que uno afirma siempre cierta.
05	AUDITORÍA EXTERNA	Ojos que no escribieron el código. Cuesta, pero es la última red antes de custodiar valor.
06	RECOMPENSAS Y MONITOREO	Bug bounties (Immunefi y otros) y vigilancia del contrato ya desplegado.

SLITHER: SALIDA REAL SOBRE EL BANCO

```
$ pip install slither-analyzer          # requiere Python 3.10+
$ slither contracts/s09/BancoVulnerable.sol
```

SALIDA REAL (REENTRANCY-ETH · SEVERIDAD ALTA)

```
Reentrancy in BancoVulnerable.retirar() (BancoVulnerable.sol#21-30):
    External calls:
    - (ok,None) = address(msg.sender).call{value: monto}()   (#26)
    State variables written after the call(s):
    - saldos[msg.sender] = 0   (#29)
Reference: .../Detector-Documentation#reentrancy-vulnerabilities-1
analyzed (1 contracts with 102 detectors), 2 result(s) found
```

⚠ QUÉ NO PUEDE HACER

Slither encontró la reentrancy en segundos y sin ejecutar nada. Pero no entiende la lógica de negocio, no sabe qué DEBERÍA hacer el contrato, y no encuentra fallos de diseño nuevos. Es el primer filtro, no el último; no reemplaza al auditor humano.

FUZZING, INVARIANTES Y AUDITORÍA EXTERNA

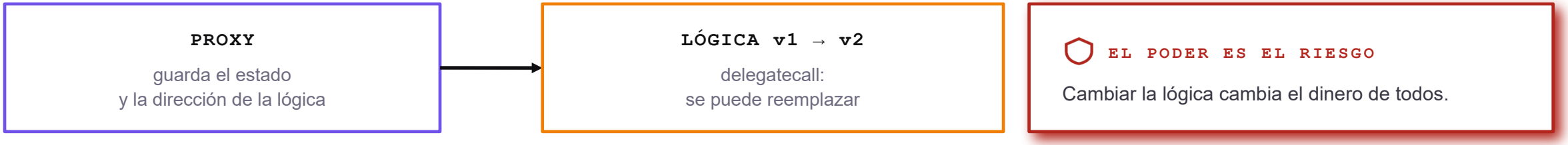
TÉCNICA	QUÉ HACE	QUÉ ATRAPA QUE LO ANTERIOR NO
Fuzzing	Lanza miles de entradas aleatorias contra el contrato buscando romper algo.	Combinaciones raras de valores que ningún test escrito a mano previó.
Invariantes	Afirmas algo que SIEMPRE debe ser cierto (p. ej. suma de saldos = balance) y se pone a prueba.	Errores de contabilidad y de diseño que no violan una función suelta.
Auditoría externa	Personas expertas que no escribieron el código lo revisan a fondo.	Fallos de lógica de negocio y de arquitectura; lo que ninguna herramienta ve.
Bug bounty	Se paga a quien encuentre fallos en el contrato YA desplegado (Immunefi y otros).	Lo que se escapó de todo lo anterior, antes de que lo use un atacante.

EL ORDEN IMPORTA

Cada capa es más cara que la anterior. Se sube en orden: no se paga una auditoría de un código que ni siquiera pasa Slither. La auditoría no es un sello: es una foto en un momento, sobre una versión concreta.

PROXIES: PODER CAMBIAR EL CÓDIGO, Y SU PRECIO

Si el código es inmutable, ¿cómo se corrige un bug? Con un proxy: un contrato «fachada» que guarda el estado y, vía delegatecall, ejecuta la lógica de OTRO contrato que sí se puede reemplazar.



EL PODER ES EL RIESGO

Cambiar la lógica cambia el dinero de todos.

LOS RIESGOS PROPIOS DEL PROXY

- Colisión de almacenamiento entre proxy y lógica.
- Un initializer sin proteger: como Parity (A.9).
- La clave de administración: si se filtra, se acabó.

CÓMO SE MITIGA

- Patrones probados de OpenZeppelin (UUPS, Transparent).
- El admin es un multisig o una DAO, no una sola llave.
- Un timelock: los cambios se anuncian antes de aplicarse.

«Actualizable» y «descentralizado» están en tensión: cada mecanismo de actualización es un poder que alguien concentra.

SEIS IDEAS ANTES DE ATACAR

01 El código es inmutable, público y custodia valor: no hay parche ni reverso.

02 call a un contrato le cede el control: puede reentrar antes de que actualices el estado.

03 Reentrancy se cierra con CEI, y se blinda con un guard. Ambos.

04 Los permisos van con msg.sender, jamás con tx.origin.

05 El azar seguro no existe on-chain: se necesita un oráculo o commit-reveal.

06 La seguridad es una cadena de capas —Slither, fuzzing, auditoría, bounties—; ninguna sola basta.

B

APROXIMADAMENTE 35 MINUTOS

ETHERNAUT + AUDITORÍA CRUZADA

Jugar a romper contratos, y después leer el de otro equipo con la mirada de quien busca la grieta.

UN CTF DONDE GANAR ES HACKEAR

Ethernaut, de OpenZeppelin, es una serie de contratos con una vulnerabilidad cada uno. Se «gana» un nivel explotándolo desde la consola del navegador. Es la mejor forma de fijar lo del bloque A.

NIVEL	VULNERABILIDAD	CON QUÉ SE CONECTA
1 · **Fallback**	Control de acceso por receive/fallback mal puestos.	S7 receive/fallback · A.8
2 · **Fallout**	Un «constructor» que en realidad es una función pública.	S6 constructor
5 · **Token**	Desbordamiento aritmético (contrato en versión 0.6).	A.10 de hoy
11 · **Re-entrancy**	El robo que acabamos de ver, para explotar con las manos.	A.3 a A.7
7 · **Force**	Enviar ETH a un contrato sin receive, con selfdestruct.	Suposiciones falsas sobre el saldo

Prerrequisito: el nivel 0 (Hello Ethernaut) quedó de trabajo de la Sesión 8. Hoy se empieza en Fallback. La guía en PDF trae el método de cada nivel.

LA CONSOLA DEL NAVEGADOR ES TU TERMINAL

01	CONECTAR	MetaMask del curso en Sepolia. Abrir la consola del navegador (F12 → Console).
02	OBTENER INSTANCIA	Botón «Get New Instance»: Ethernaut despliega un contrato solo para ti y lo expone como `contract`.
03	INSPECCIONAR	`player`, `await contract.info()`, y sobre todo `contract.abi`: ahí están TODAS las funciones que puedes llamar.
04	INTERACTUAR	`await contract.metodo(args)` o `contract.sendTransaction({value: ...})` para enviar ETH.
05	ENVIAR	Botón «Submit instance»: Ethernaut comprueba si cumpliste el objetivo del nivel.

[-] EL REFLEJO QUE HAY QUE ADQUIRIR

Ante cualquier nivel: mirar primero contract.abi y el código fuente. La vulnerabilidad casi siempre salta leyendo.

LEER EL CONTRATO DE OTRO EQUIPO

Cada equipo entrega el contrato principal de su proyecto y recibe el de otro. Media hora para auditarlo con una lista concreta. Leer código ajeno es parte de la competencia.

01	RECORRER LA TAXONOMÍA	Las ocho vulnerabilidades de A.2, una por una, contra el contrato recibido.
02	PASAR SLITHER	Correrlo y leer los hallazgos: separar los reales de los falsos positivos.
03	CLASIFICAR	Cada hallazgo con severidad —crítica, alta, media, baja, informativa— y la línea exacta.
04	PROPONER	Para cada uno, la corrección mínima. No reescribir el contrato ajeno.

EL FORMATO Y LA RÚBRICA ESTÁN EN EL MATERIAL DEL PROYECTO

«Auditoría cruzada» (HTML + PDF, público) trae el formato de hallazgo, cómo se asignan los equipos, qué se entrega y la rúbrica. Se califica la calidad de la auditoría, no cuántos fallos tenga el otro.

C

90 MINUTOS · 45 % DEL SEGUNDO CORTE

PARCIAL PRÁCTICO

Un contrato vulnerable. Encontrar los fallos, explotar al menos uno y entregar la versión corregida. 90 minutos, individual, libro abierto.

LAS TRES TAREAS DEL PARCIAL

	DETALLE
El insumo	Un contrato vulnerable, entregado al empezar el bloque, con vulnerabilidades de la taxonomía de hoy.
Tarea 1 · identificar	Nombrar cada fallo, con su severidad y su línea exacta.
Tarea 2 · explotar	Una prueba en Hardhat que demuestre al menos un abuso.
Tarea 3 · corregir	La versión corregida, sin romper lo legítimo y sin reescribir el contrato entero.
Se entrega	El contrato corregido, la prueba del exploit y media página con los hallazgos y sus correcciones.

UNA RESTRICCIÓN IMPORTANTE

Corregir lo MÍNIMO necesario: reescribir el contrato entero cuesta puntos. Un evento que falta, una pausa ausente o una contabilidad que no cuadra son mejoras o bugs, no vulnerabilidades de seguridad: no cuentan como hallazgo.

TIEMPOS, REGLAS Y USO DE IA

	DETALLE
Duración	90 minutos exactos, en máquina, en el repositorio de laboratorios del curso.
Modalidad	Individual. Cada quien entrega lo suyo.
Libro abierto	Material del curso, documentación oficial y el repositorio. Todo lo escrito está permitido.
Sobre la IA	Sin asistentes de IA (Copilot, ChatGPT y similares) ni ayuda de otra persona. El objetivo es evaluar TU criterio de seguridad.
Peso	45 % del segundo corte.
Integridad	Una sospecha fundada anula el parcial. Se revisa el historial de escritura si hace falta.

Sugerencia de reparto: ~25 min leer e identificar, ~35 min escribir el exploit, ~25 min corregir, ~5 min redactar la media página.

RÚBRICA SOBRE 20 PUNTOS

CRITERIO	PUNTOS	QUÉ SE ESPERA
Identificar los fallos	6	Nombre correcto y línea. Media si nombra sin ubicar.
Severidad justificada	3	Coherente con el impacto real de cada una.
Exploit funcional	5	Una prueba que demuestra el abuso. Más si roba fondos.
Corregir los fallos	4	Todos, sin romper la funcionalidad legítima.
No romper lo legítimo	2	Prueba de humo del docente sobre el contrato corregido.

[-] CÓMO SUBIR LA NOTA

La severidad se justifica con el impacto: ¿se roban fondos, se bloquea el contrato, o solo es una molestia? Un exploit que efectivamente MUEVE dinero vale más que uno que solo describe el abuso. Y la corrección mínima que conserva depositar/retirar honestos es la mitad de la nota de corrección.

LA PREGUNTA DE LA SEMANA

De las ocho vulnerabilidades de hoy, ¿cuáles podría tener SU proyecto? Revísenlo con la lista en la mano antes del Avance 1.

PARA LA PRÓXIMA SESIÓN

- Terminar los cinco niveles de Ethernaut si no se alcanzaron en clase.
- Entregar el informe de la auditoría cruzada #1 con el formato del material del proyecto.
- Pasar Slither al contrato del proyecto y anotar los hallazgos.
- Empezar la Unidad III: leer sobre el estándar ERC-20 para la Sesión 10.
- Nivelación de React: quien no lo haya visto, empezar el tutorial entregado. La Sesión 12 lo asume.

LO QUE SE LLEVAN DE HOY

Para escribir un contrato que nadie pueda robar, primero hay que aprender a robarlo.

Con esto cierra la Unidad II: ya saben escribir, probar, desplegar y auditar. La Unidad III construye cosas que la gente usa: tokens, NFTs y una interfaz de verdad.

SESIÓN 10 · TOKENS FUNGIBLES · EL ESTÁNDAR ERC-20